

First 20 — Full Solutions

Striver A2Z · Step 1: Learn the Basics · In Depth · C++ & Python

What's inside. Every one of the first 20 items, solved in **both C++ and Python**, each with the approach, the full code, and a complexity note. The first 12 are language/STL warm-ups shown with concrete runnable demos; items 13–20 are the real coding problems (maths + recursion). All code was compiled with `g++ -std=c++17` and run in Python before printing.

How to use it: read the Approach, try to write both versions yourself, then compare. The C++ block is on top, Python below, so you can see how the same idea maps between the two languages.

Q1. User Input / Output

■ **Approach:** Read values from standard input, do something, print the result. In C++ use cin/cout; in Python use input()/print(). Always convert input to int/float when you need numbers.

C++

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int a, b;
    cin >> a >> b;          // read two ints
    cout << a + b << "\n";  // print the sum
    return 0;
}
```

PYTHON

```
a = int(input())    # read & convert to int
b = int(input())
print(a + b)        # print the sum
```

Complexity & notes: Reading input is $O(1)$ per value. In C++, add `ios_base::sync_with_stdio(false); cin.tie(NULL);` at the top of `main()` to speed up cin/cout for large inputs.

Q2. Data Types

■ **Approach:** Pick a type that fits your value's size. C++ is strict about types and ranges; Python ints grow without limit. The classic trap: an int overflowing in C++ when a product or sum exceeds ~ 2.1 billion — use long long.

C++

```
int    i = 42;          // ~ +/- 2.1 x 10^9
long long big = 1e18;   // huge whole numbers
double d = 3.14;        // decimals
char   c = 'A';
bool   ok = true;
string s = "hi";
```

PYTHON

```
i = 42          # unbounded int
d = 3.14        # float
c = "A"         # single-char string
ok = True       # bool
s = "hi"        # str
```

Complexity & notes: Know the int limit in C++ ($\sim 2.1 \times 10^9$). If a calculation might exceed it, use long long. Python has no overflow, so this trap only bites in C++.

Q3. If-Else Statements

■ **Approach:** Branch on a condition. Use if / else if / else. Keep conditions simple and mutually exclusive where possible.

C++

```
int n = 0;
if (n > 0)
    cout << "positive";
else if (n < 0)
    cout << "negative";
else
    cout << "zero";
```

PYTHON

```
n = 0
if n > 0:
    print("positive")
elif n < 0:
    print("negative")
else:
    print("zero")
```

Complexity & notes: O(1). Python uses elif; C++ uses else if. Indentation defines blocks in Python; braces { } in C++.

Q4. Switch Statement

■ **Approach:** A clean multi-way branch on one variable's value. C++ has a real switch (remember break; after each case, or execution 'falls through'). Python has no switch — use if/elif or a dictionary mapping.

C++

```
switch (day) {
    case 1: cout << "Mon"; break;
    case 2: cout << "Tue"; break;
    default: cout << "Other";
}
```

PYTHON

```
day = 1
# dict acts like switch
names = {1: "Mon", 2: "Tue"}
print(names.get(day, "Other"))
```

Complexity & notes: Forgetting break; in C++ is the classic switch bug — cases fall through to the next. Python's dict.get(key, default) is the idiomatic■■■.

Q5. Arrays & Strings Basics

■ **Approach:** Store many values in order, index from 0. C++ uses vector (dynamic) or arrays; Python uses list. Strings are sequences of characters in both.

C++

```
vector<int> a = {10, 20, 30};
a.push_back(40);           // add at end
cout << a[0] << a.size();
string s = "code";
cout << s[0] << s.size();
```

PYTHON

```
a = [10, 20, 30]
a.append(40)               # add at end
print(a[0], len(a))
s = "code"
print(s[0], len(s))
```

Complexity & notes: Indexing/append are $O(1)$. Both are 0-indexed. C++ `vector.size()` and `string.size()`; Python `len()`.

Q6. For Loops

■ **Approach:** Repeat a fixed number of times or iterate a container. C++ uses the classic `for(init; cond; step)`; Python's `for` iterates a range or a collection directly.

C++

```
for (int i = 0; i < 5; i++)
    cout << i << " ";           // 0 1 2 3 4

vector<int> v = {1,2,3};
for (int x : v)                 // range-for
    cout << x;
```

PYTHON

```
for i in range(5):
    print(i, end=" ")           # 0 1 2 3 4

v = [1, 2, 3]
for x in v:                     # iterate items
    print(x)
```

Complexity & notes: $O(n)$ for n iterations. `range(5)` gives 0..4. C++ range-for (`for(int x : v)`) is the closest match to Python's `for x in v`.

Q7. While Loops

■ **Approach:** Repeat while a condition holds. Use when you don't know the count up front (e.g. 'until n becomes 0'). Make sure something inside changes the condition or you loop forever.

C++

```
int n = 8;
while (n > 1) {
    cout << n << " ";
    n = n / 2;          // changes n
}
```

PYTHON

```
n = 8
while n > 1:
    print(n, end=" ")
    n = n // 2          # changes n
```

Complexity & notes: The body must move toward making the condition false. A while whose variables never change is an infinite loop (a real bug you hit in binary search).

Q8. Functions — Pass by Reference vs Value

■ **Approach:** By value copies the argument (caller's variable is untouched). By reference shares it (changes persist). In C++ add & to a parameter to pass by reference. In Python, mutable objects (list/dict) behave reference-like; numbers/strings behave value-like.

C++

```
void byValue(int x){ x = 99; }      // no effect outside
void byRef(int& x){ x = 99; }      // changes caller

int a = 1; byValue(a); // a still 1
int b = 1; byRef(b);   // b now 99
```

PYTHON

```
def add_item(lst):                # list is shared
    lst.append(99)                 # caller sees it

def reassign(x):                  # int is value-like
    x = 99                        # no effect outside
a = [1]; add_item(a)              # a == [1, 99]
```

Complexity & notes: Pass by reference avoids copying big objects (faster) and lets a function modify the caller's data. In C++ use const& to pass big objects fast WITHOUT allowing changes.

Q9. Time & Space Complexity

■ **Approach:** Time = how the number of operations grows with input size n . Space = extra memory used. Judge every solution by its Big-O. The example below contrasts $O(n)$ (one loop) with $O(n^2)$ (nested loops).

C++

```
// O(n): one pass
long sum = 0;
for (int i = 0; i < n; i++) sum += a[i];

// O(n^2): nested
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        cout << a[i] + a[j];
```

PYTHON

```
# O(n): one pass
total = sum(a)

# O(n^2): nested
for i in range(n):
    for j in range(n):
        print(a[i] + a[j])
```

Complexity & notes: Order to remember: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$. A nested loop over the same array is the #1 source of accidental $O(n^2)$.

Q10. Patterns — Nested-Loop Thinking

■ **Approach:** Outer loop = the current row; inner loop(s) = how many symbols print in that row. For each pattern, just find: how many symbols and how many leading spaces in row i . Below: a triangle, a number triangle, and a centered pyramid.

C++

```
// 1) right triangle of *      2) number triangle      3) pyramid
void triangle(int n){
    for(int i=1;i<=n;i++){          // row i
        for(int j=0;j<i;j++) cout<<"*"; // i stars
        cout<<"\n"; } }
void numTri(int n){
    for(int i=1;i<=n;i++){
        for(int j=1;j<=i;j++) cout<<j; // 1..i
        cout<<"\n"; } }
void pyramid(int n){
    for(int i=1;i<=n;i++){
        for(int s=0;s<n-i;s++) cout<<" "; // leading spaces
        for(int j=0;j<2*i-1;j++) cout<<"*"; // 2i-1 stars
        cout<<"\n"; } }
```

PYTHON

```
def triangle(n):
    for i in range(1, n+1):
        print(" " * i)          # i stars

def num_tri(n):
    for i in range(1, n+1):
        print("".join(str(j) for j in range(1, i+1)))

def pyramid(n):
    for i in range(1, n+1):
        print(" "*(n-i) + ""*(2*i-1)) # spaces + stars
```

Complexity & notes: Each pattern is $O(\text{rows} \times \text{cols})$. The skill isn't memorizing them — it's deriving 'symbols in row i' and 'spaces in row i'. Do 5-6 from Striver's video using that recipe.

Q11. C++ STL — Your Toolbox

■ **Approach:** The Standard Template Library gives ready-made containers so you never hand-build a stack, map or heap. These appear in nearly every later problem. Python's built-ins are the equivalents.

C++

```
vector<int> v = {3,1,2};
sort(v.begin(), v.end());           // -> 1 2 3
map<string,int> freq;                 // ordered key->val
freq["a"]++;
set<int> s = {1,1,2};                 // unique -> {1,2}
stack<int> st; st.push(5);
queue<int> q; q.push(5);
priority_queue<int> pq;               // max-heap
```

PYTHON

```
v = [3, 1, 2]
v.sort()                               # -> [1,2,3]
freq = {}                              # dict key->val
freq["a"] = freq.get("a",0)+1
s = set([1, 1, 2])                     # -> {1,2}
st = []; st.append(5)                  # stack
from collections import deque          # queue
import heapq                           # heap
```

Complexity & notes: Learn these names now — calling `sort()` / a `map` / a `priority_queue` beats re-implementing them. You're doing Python + C++, so skip Java Collections (item 12).

Q12. Java Collections (skip — you use C++ & Python)

■ **Approach:** This item is Java's version of the STL (ArrayList, HashMap, HashSet, etc.). Since you're solving in C++ and Python, you can safely skip it — the STL table in Q11 already covers the same tools. Shown here only for completeness.

C++

```
// C++ equivalents (use these instead):  
// ArrayList -> vector  
// HashMap   -> unordered_map  
// HashSet    -> unordered_set  
// Stack      -> stack  
// PriorityQueue -> priority_queue
```

PYTHON

```
# Python equivalents:  
# ArrayList -> list  
# HashMap   -> dict  
# HashSet    -> set  
# Stack      -> list  
# PriorityQueue -> heapq
```

Complexity & notes: Nothing to solve here. Tick it and move on to the maths problems.

Q13. Count Digits

Easy

■ **Approach:** Chop the last digit repeatedly with $n/10$ and count how many chops until n hits 0. $n\%10$ is the last digit; $n/10$ removes it. Edge case: 0 has 1 digit.

C++

```
int countDigits(int n){
    n = abs(n);
    if(n == 0) return 1;      // edge case
    int c = 0;
    while(n > 0){
        c++;                  // one more digit
        n /= 10;              // drop last digit
    }
    return c;
}
```

PYTHON

```
def count_digits(n):
    n = abs(n)
    if n == 0: return 1      # edge case
    c = 0
    while n > 0:
        c += 1               # one more digit
        n //= 10             # drop last digit
    return c
```

Complexity & notes: $O(\log_{10} n)$ — the number of digits. Alternative one-liner: `len(str(abs(n)))` in Python, or $(\text{int})\log_{10}(n)+1$ mathematically.

Q14. Reverse a Number

Easy

■ **Approach:** Build the reverse digit-by-digit: pull the last digit with $n\%10$ and append it to rev via $rev = rev*10 + \text{digit}$, then drop it with $n/10$. Track the sign separately.

C++

```
int reverseNumber(int n){
    int sign = n < 0 ? -1 : 1;
    n = abs(n);
    int rev = 0;
    while(n > 0){
        rev = rev*10 + n%10;    // append last digit
        n /= 10;
    }
    return sign * rev;
}
```

PYTHON

```
def reverse_number(n):
    sign = -1 if n < 0 else 1
    n = abs(n); rev = 0
    while n > 0:
        rev = rev*10 + n % 10    # append last digit
        n //= 10
    return sign * rev
```

Complexity & notes: $O(\log_{10} n)$. In C++ watch for overflow if the reversed value exceeds int range — use long long for very large inputs.

Q15. Check Palindrome (Number)

Easy

■ **Approach:** A number is a palindrome if it reads the same reversed. Reuse the reverse logic and compare to the original. (For strings, the two-pointer method compares ends moving inward.)

C++

```
int rev(int n){
    int r = 0;
    while(n > 0){ r = r*10 + n%10; n /= 10; }
    return r;
}
bool isPalindrome(int n){
    return n >= 0 && n == rev(n);
}
```

PYTHON

```
def rev(n):
    r = 0
    while n > 0:
        r = r*10 + n % 10; n //= 10
    return r

def is_palindrome(n):
    return n >= 0 and n == rev(n)
```

Complexity & notes: $O(\log_{10} n)$. Negative numbers are not palindromes (the minus sign breaks symmetry). String way in Python: `str(n) == str(n)[::-1]`.

Q16. GCD / HCF

Easy

■ **Approach:** Euclidean algorithm: $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$, repeating until b becomes 0 — then a holds the GCD. Intuition: any common divisor of a and b also divides $a \bmod b$, so the problem shrinks fast.

C++

```
int gcd(int a, int b){
    while(b){
        int t = a % b;
        a = b;
        b = t;           // shrink toward 0
    }
    return a;           // b==0 -> a is gcd
}
```

PYTHON

```
def gcd(a, b):
    while b:
        a, b = b, a % b    # shrink toward 0
    return a               # b==0 -> a is gcd
```

Complexity & notes: $O(\log(\min(a,b)))$ — very fast. $\text{LCM}(a,b) = a*b / \text{gcd}(a,b)$ (use long long in C++ to avoid overflow). C++17 also has `std::gcd`.

Q17. Armstrong Number

Easy

■ **Approach:** A k -digit number is Armstrong if the sum of each digit raised to power k equals the number. 153 has 3 digits and $1^3 + 5^3 + 3^3 = 153$. Find k (digit count), then sum the powers.

C++

```
bool isArmstrong(int n){
    string s = to_string(n);
    int k = s.size();
    long sum = 0;
    for(char c : s)
        sum += pow(c - '0', k); // digit^k
    return n == sum;
}
```

PYTHON

```
def is_armstrong(n):
    s = str(n); k = len(s)
    total = sum(int(d)**k for d in s)
    return n == total
```

Complexity & notes: $O(k)$ where k = number of digits. $c - '0'$ converts a char digit to its int value in C++. In Python, `int(d)` does the same.

Q18. Print All Divisors

Easy

■ **Approach:** Don't loop to n . Loop i only to $\sqrt{n}$: whenever i divides n , BOTH i and n/i are divisors. Collect both (avoid double-counting when $i == n/i$, e.g. for perfect squares). Drops $O(n)$ to $O(\sqrt{n})$.

C++

```
vector<int> divisors(int n){
    vector<int> res;
    for(int i = 1; (long)i*i <= n; i++){
        if(n % i == 0){
            res.push_back(i);
            if(i != n/i) // avoid dup
                res.push_back(n/i);
        }
    }
    sort(res.begin(), res.end());
    return res;
}
```

PYTHON

```
def divisors(n):
    res = []
    i = 1
    while i*i <= n:
        if n % i == 0:
            res.append(i)
            if i != n // i: # avoid dup
                res.append(n // i)
        i += 1
    return sorted(res)
```

Complexity & notes: $O(\sqrt{n})$ to find them, plus $O(d \log d)$ to sort the d divisors. The $\sqrt{n}$ pairing trick is reused everywhere (prime check, factorization).

Q19. Check for Prime

Easy

■ **Approach:** A prime has exactly two divisors: 1 and itself. So test divisibility only up to $\sqrt{n}$ — if nothing in $[2, \sqrt{n}]$ divides n , it's prime. Same $\sqrt{n}$ idea as divisors. Handle $n < 2$ as not prime.

C++

```
bool isPrime(int n){
    if(n < 2) return false;
    for(int i = 2; (long)i*i <= n; i++)
        if(n % i == 0)
            return false;    // found a factor
    return true;
}
```

PYTHON

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i*i <= n:
        if n % i == 0:
            return False    # found a factor
        i += 1
    return True
```

Complexity & notes: $O(\sqrt{n})$ per check. To test MANY numbers up to N , use the Sieve of Eratosthenes for $O(N \log \log N)$ total (you'll meet it in the Bit Manipulation step).

Q20. Print Name N Times (Recursion)

Easy

■ **Approach:** Your first recursion. Base case: when n hits 0, stop (return). Recursive case: do the work once (print), then call yourself with $n-1$. Each call moves toward the base case, so it always terminates.

C++

```
void printName(string name, int n){
    if(n == 0) return;        // base case: stop
    cout << name << "\n";    // do work once
    printName(name, n - 1);   // smaller problem
}
// call: printName("Boss", 5);
```

PYTHON

```
def print_name(name, n):
    if n == 0:                # base case: stop
        return
    print(name)               # do work once
    print_name(name, n - 1)   # smaller problem

print_name("Boss", 5)
```

Complexity & notes: $O(n)$ time and $O(n)$ call-stack space (n stacked calls before they return). This base-case + smaller-call shape is the template for ALL recursion ahead.

That's all 20 in both languages. The reusable wins from Day 1: the **sqrt(n) trick** (divisors & primes), the **Euclidean GCD**, the **digit loop** ($n\%10$ / $n//10$), and the **recursion template** (base case + smaller call). Notice how cleanly each idea maps between C++ and Python — the logic is identical, only the syntax differs. Re-type each from a blank page in both languages, tick them in your tracker, and you're ready for Day 2 (Sorting + Arrays).